

SKILL

Checklist Purpose: “Unit Tests for English”

CRITICAL CONCEPT: Checklists are **UNIT TESTS FOR REQUIREMENTS WRITING** - they validate the quality, clarity, and completeness of requirements in a given domain.

NOT for verification/testing:

- NOT “Verify the button clicks correctly”
- NOT “Test error handling works”
- NOT “Confirm the API returns 200”
- NOT checking if code/implementation matches the spec

FOR requirements quality validation:

- “Are visual hierarchy requirements defined for all card types?” (completeness)
- “Is ‘prominent display’ quantified with specific sizing/positioning?” (clarity)
- “Are hover state requirements consistent across all interactive elements?” (consistency)
- “Are accessibility requirements defined for keyboard navigation?” (coverage)
- “Does the spec define what happens when logo image fails to load?” (edge cases)

Metaphor: If your spec is code written in English, the checklist is its unit test suite. You’re testing whether the requirements are well-written, complete, unambiguous, and ready for implementation - NOT whether the implementation works.

User Input

\$ARGUMENTS

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before checklist generation): -

Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.before_checklist` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (.) with hyphens (-). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its optional flag: - **Optional hook** (`optional: true`): `` ` ## Extension Hooks

```
**Optional Pre-Hook**: {extension}
```

```
Command: `{command}`
```

```
Description: {description}
```

```
Prompt: {prompt}
```

```
To execute: `{command}`
```

```
`` `
```

- **Mandatory hook** (`optional: false`):

```
## Extension Hooks
```

```
**Automatic Pre-Hook**: {extension}
```

```
Executing: `{command}`
```

```
EXECUTE_COMMAND: {command}
```

Wait for the result of the hook command before proceeding to the Execution Steps.

After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

Execution Steps

1. **Setup:** Run `.specify/scripts/bash/check-prerequisites.sh --json` from repo root and parse JSON for `FEATURE_DIR` and `AVAILABLE_DOCS` list.
 - All file paths must be absolute.
 - For single quotes in args like "I'm Groot", use escape syntax: e.g 'I''m Groot' (or double-quote if possible: "I'm Groot").
2. **IF EXISTS:** Load `.specify/memory/constitution.md` for project principles and governance constraints.
3. **Clarify intent (dynamic):** Derive up to THREE initial contextual clarifying questions (no pre-baked catalog). They MUST:
 - Be generated from the user's phrasing + extracted signals from spec/plan/tasks
 - Only ask about information that materially changes checklist content
 - Be skipped individually if already unambiguous in `$ARGUMENTS`
 - Prefer precision over breadth

Generation algorithm:

1. Extract signals: feature domain keywords (e.g., auth, latency, UX, API), risk indicators ("critical", "must", "compliance"), stakeholder hints ("QA", "review", "security team"), and explicit deliverables ("a11y", "rollback", "contracts").
2. Cluster signals into candidate focus areas (max 4) ranked by relevance.
3. Identify probable audience & timing (author, reviewer, QA, release) if not explicit.
4. Detect missing dimensions: scope breadth, depth/rigor, risk emphasis, exclusion boundaries, measurable acceptance criteria.
5. Formulate questions chosen from these archetypes:
 - Scope refinement (e.g., "Should this include integration touchpoints with X and Y or stay limited to local module correctness?")
 - Risk prioritization (e.g., "Which of these potential risk areas should receive mandatory gating checks?")
 - Depth calibration (e.g., "Is this a lightweight pre-commit sanity list or a formal release gate?")
 - Audience framing (e.g., "Will this be used by the author only or peers during PR review?")
 - Boundary exclusion (e.g., "Should we explicitly exclude performance tuning items this round?")

- Scenario class gap (e.g., “No recovery flows detected—are rollback / partial failure paths in scope?”)

Question formatting rules:

- If presenting options, generate a compact table with columns: Option | Candidate | Why It Matters
- Limit to A-E options maximum; omit table if a free-form answer is clearer
- Never ask the user to restate what they already said
- Avoid speculative categories (no hallucination). If uncertain, ask explicitly: “Confirm whether X belongs in scope.”

Defaults when interaction impossible:

- Depth: Standard
- Audience: Reviewer (PR) if code-related; Author otherwise
- Focus: Top 2 relevance clusters

Output the questions (label Q1/Q2/Q3). After answers: if ≥ 2 scenario classes (Alternate / Exception / Recovery / Non-Functional domain) remain unclear, you MAY ask up to TWO more targeted follow-ups (Q4/Q5) with a one-line justification each (e.g., “Unresolved recovery path risk”). Do not exceed five total questions. Skip escalation if user explicitly declines more.

4. Understand user request: Combine \$ARGUMENTS + clarifying answers:

- Derive checklist theme (e.g., security, review, deploy, ux)
- Consolidate explicit must-have items mentioned by user
- Map focus selections to category scaffolding
- Infer any missing context from spec/plan/tasks (do NOT hallucinate)

5. Load feature context: Read from FEATURE_DIR:

- spec.md: Feature requirements and scope
- plan.md (if exists): Technical details, dependencies
- tasks.md (if exists): Implementation tasks

Context Loading Strategy:

- Load only necessary portions relevant to active focus areas (avoid full-file dumping)
- Prefer summarizing long sections into concise scenario/requirement bullets
- Use progressive disclosure: add follow-on retrieval only if gaps detected
- If source docs are large, generate interim summary items instead of embedding raw text

6. **Generate checklist** - Create “Unit Tests for Requirements”:

- Create FEATURE_DIR/checklists/ directory if it doesn't exist
- Generate unique checklist filename:
 - Use short, descriptive name based on domain (e.g., ux.md, api.md, security.md)
 - Format: [domain].md
- File handling behavior:
 - If file does NOT exist: Create new file and number items starting from CHK001
 - If file exists: Append new items to existing file, continuing from the last CHK ID (e.g., if last item is CHK015, start new items at CHK016)
- Never delete or replace existing checklist content - always preserve and append

CORE PRINCIPLE - Test the Requirements, Not the Implementation: Every checklist item MUST evaluate the REQUIREMENTS THEMSELVES for:

- **Completeness:** Are all necessary requirements present?
- **Clarity:** Are requirements unambiguous and specific?
- **Consistency:** Do requirements align with each other?
- **Measurability:** Can requirements be objectively verified?
- **Coverage:** Are all scenarios/edge cases addressed?

Category Structure - Group items by requirement quality dimensions:

- **Requirement Completeness** (Are all necessary requirements documented?)
- **Requirement Clarity** (Are requirements specific and unambiguous?)
- **Requirement Consistency** (Do requirements align without conflicts?)
- **Acceptance Criteria Quality** (Are success criteria measurable?)
- **Scenario Coverage** (Are all flows/cases addressed?)
- **Edge Case Coverage** (Are boundary conditions defined?)
- **Non-Functional Requirements** (Performance, Security, Accessibility, etc. - are they specified?)
- **Dependencies & Assumptions** (Are they documented and validated?)
- **Ambiguities & Conflicts** (What needs clarification?)

HOW TO WRITE CHECKLIST ITEMS - “Unit Tests for English”:

WRONG (Testing implementation):

- “Verify landing page displays 3 episode cards”
- “Test hover states work on desktop”
- “Confirm logo click navigates home”

CORRECT (Testing requirements quality):

- “Are the exact number and layout of featured episodes specified?” [Completeness]
- “Is ‘prominent display’ quantified with specific sizing/positioning?” [Clarity]
- “Are hover state requirements consistent across all interactive elements?” [Consistency]
- “Are keyboard navigation requirements defined for all interactive UI?” [Coverage]
- “Is the fallback behavior specified when logo image fails to load?” [Edge Cases]
- “Are loading states defined for asynchronous episode data?” [Completeness]
- “Does the spec define visual hierarchy for competing UI elements?” [Clarity]

ITEM STRUCTURE: Each item should follow this pattern:

- Question format asking about requirement quality
- Focus on what’s WRITTEN (or not written) in the spec/plan
- Include quality dimension in brackets [Completeness/Clarity/Consistency/etc.]
- Reference spec section [Spec §X.Y] when checking existing requirements
- Use [Gap] marker when checking for missing requirements

EXAMPLES BY QUALITY DIMENSION:

Completeness:

- “Are error handling requirements defined for all API failure modes? [Gap]”
- “Are accessibility requirements specified for all interactive elements? [Completeness]”
- “Are mobile breakpoint requirements defined for responsive layouts? [Gap]”

Clarity:

- “Is ‘fast loading’ quantified with specific timing thresholds? [Clarity, Spec §NFR-2]”
- “Are ‘related episodes’ selection criteria explicitly defined? [Clarity, Spec §FR-5]”
- “Is ‘prominent’ defined with measurable visual properties? [Ambiguity, Spec §FR-4]”

Consistency:

- “Do navigation requirements align across all pages? [Consistency, Spec §FR-10]”
- “Are card component requirements consistent between landing and detail pages? [Consistency]”

Coverage:

- “Are requirements defined for zero-state scenarios (no episodes)? [Coverage, Edge Case]”
- “Are concurrent user interaction scenarios addressed? [Coverage, Gap]”
- “Are requirements specified for partial data loading failures? [Coverage, Exception Flow]”

Measurability:

- “Are visual hierarchy requirements measurable/testable? [Acceptance Criteria, Spec §FR-1]”
- “Can ‘balanced visual weight’ be objectively verified? [Measurability, Spec §FR-2]”

Scenario Classification & Coverage (Requirements Quality Focus):

- Check if requirements exist for: Primary, Alternate, Exception/Error, Recovery, Non-Functional scenarios
- For each scenario class, ask: “Are [scenario type] requirements complete, clear, and consistent?”
- If scenario class missing: “Are [scenario type] requirements intentionally excluded or missing? [Gap]”
- Include resilience/rollback when state mutation occurs: “Are rollback requirements defined for migration failures? [Gap]”

Traceability Requirements:

- MINIMUM: $\geq 80\%$ of items MUST include at least one traceability reference
- Each item should reference: spec section [Spec §X.Y], or use markers: [Gap], [Ambiguity], [Conflict], [Assumption]
- If no ID system exists: “Is a requirement & acceptance criteria ID scheme established? [Traceability]”

Surface & Resolve Issues (Requirements Quality Problems):

Ask questions about the requirements themselves:

- Ambiguities: “Is the term ‘fast’ quantified with specific metrics? [Ambiguity, Spec §NFR-1]”
- Conflicts: “Do navigation requirements conflict between §FR-10 and §FR-10a? [Conflict]”
- Assumptions: “Is the assumption of ‘always available podcast API’ validated? [Assumption]”
- Dependencies: “Are external podcast API requirements documented? [Dependency, Gap]”
- Missing definitions: “Is ‘visual hierarchy’ defined with measurable criteria? [Gap]”

Content Consolidation:

- Soft cap: If raw candidate items > 40, prioritize by risk/impact
- Merge near-duplicates checking the same requirement aspect
- If >5 low-impact edge cases, create one item: "Are edge cases X, Y, Z addressed in requirements? [Coverage]"

ABSOLUTELY PROHIBITED - These make it an implementation test, not a requirements test:

- Any item starting with "Verify", "Test", "Confirm", "Check" + implementation behavior
- References to code execution, user actions, system behavior
- "Displays correctly", "works properly", "functions as expected"
- "Click", "navigate", "render", "load", "execute"
- Test cases, test plans, QA procedures
- Implementation details (frameworks, APIs, algorithms)

REQUIRED PATTERNS - These test requirements quality:

- "Are [requirement type] defined/specified/documented for [scenario]?"
- "Is [vague term] quantified/clarified with specific criteria?"
- "Are requirements consistent between [section A] and [section B]?"
- "Can [requirement] be objectively measured/verified?"
- "Are [edge cases/scenarios] addressed in requirements?"
- "Does the spec define [missing aspect]?"

7. **Structure Reference:** Generate the checklist following the canonical template in `.specify/templates/checklist-template.md` for title, meta section, category headings, and ID formatting. If template is unavailable, use: H1 title, purpose/created meta lines, ## category sections containing
- [] CHK### <requirement item> lines with globally incrementing IDs starting at CHK001.

8. **Report:** Output full path to checklist file, item count, and summarize whether the run created a new file or appended to an existing one. Summarize:

- Focus areas selected
- Depth level
- Actor/timing
- Any explicit user-specified must-have items incorporated

Important: Each `/skill:speckit-checklist` command invocation uses a short, descriptive checklist filename and either creates a new file or appends to an existing one. This allows:

- Multiple checklists of different types (e.g., `ux.md`, `test.md`, `security.md`)
- Simple, memorable filenames that indicate checklist purpose
- Easy identification and navigation in the `checklists/` folder

To avoid clutter, use descriptive types and clean up obsolete checklists when done.

Example Checklist Types & Sample Items

UX Requirements Quality: `ux.md`

Sample items (testing the requirements, NOT the implementation):

- “Are visual hierarchy requirements defined with measurable criteria? [Clarity, Spec §FR-1]”
- “Is the number and positioning of UI elements explicitly specified? [Completeness, Spec §FR-1]”
- “Are interaction state requirements (hover, focus, active) consistently defined? [Consistency]”
- “Are accessibility requirements specified for all interactive elements? [Coverage, Gap]”
- “Is fallback behavior defined when images fail to load? [Edge Case, Gap]”
- “Can ‘prominent display’ be objectively measured? [Measurability, Spec §FR-4]”

API Requirements Quality: `api.md`

Sample items:

- “Are error response formats specified for all failure scenarios? [Completeness]”
- “Are rate limiting requirements quantified with specific thresholds? [Clarity]”
- “Are authentication requirements consistent across all endpoints? [Consistency]”
- “Are retry/timeout requirements defined for external dependencies? [Coverage, Gap]”
- “Is versioning strategy documented in requirements? [Gap]”

Performance Requirements Quality: `performance.md`

Sample items:

- “Are performance requirements quantified with specific metrics? [Clarity]”
- “Are performance targets defined for all critical user journeys? [Coverage]”
- “Are performance requirements under different load conditions specified? [Completeness]”
- “Can performance requirements be objectively measured? [Measurability]”
- “Are degradation requirements defined for high-load scenarios? [Edge Case, Gap]”

Security Requirements Quality: security.md

Sample items:

- “Are authentication requirements specified for all protected resources? [Coverage]”
- “Are data protection requirements defined for sensitive information? [Completeness]”
- “Is the threat model documented and requirements aligned to it? [Traceability]”
- “Are security requirements consistent with compliance obligations? [Consistency]”
- “Are security failure/breach response requirements defined? [Gap, Exception Flow]”

Anti-Examples: What NOT To Do

WRONG - These test implementation, not requirements:

- [] CHK001 - Verify landing page displays 3 episode cards [Spec \$FR-001]
- [] CHK002 - Test hover states work correctly on desktop [Spec \$FR-003]
- [] CHK003 - Confirm logo click navigates to home page [Spec \$FR-010]
- [] CHK004 - Check that related episodes section shows 3-5 items [Spec \$FR-005]

CORRECT - These test requirements quality:

- [] CHK001 - Are the number and layout of featured episodes explicitly specified? [Completeness, Spec \$FR-001]
- [] CHK002 - Are hover state requirements consistently defined for all interactive elements? [Consistency, Spec \$FR-003]
- [] CHK003 - Are navigation requirements clear for all clickable brand elements? [Clarity, Spec \$FR-010]
- [] CHK004 - Is the selection criteria for related episodes documented? [Gap, Spec \$FR-005]

- [] CHK005 - Are loading state requirements defined for asynchronous episode data? [\[Gap\]](#)
- [] CHK006 - Can "visual hierarchy" requirements be objectively measured? [\[Measurability, Spec \\$FR-001\]](#)

Key Differences:

- Wrong: Tests if the system works correctly
- Correct: Tests if the requirements are written correctly
- Wrong: Verification of behavior
- Correct: Validation of requirement quality
- Wrong: "Does it do X?"
- Correct: "Is X clearly specified?"

Post-Execution Checks

Check for extension hooks (after checklist generation):

Check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.after_checklist` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly false. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the HookExecutor implementation - When constructing slash commands from hook command names, replace dots (.) with hyphens (-). For example, `speckit.git.commit` → `/skill:speckit-git-commit`. - For each executable hook, output the following based on its optional flag: - **Optional hook** (`optional: true`): ```` ## Extension Hooks`

```
**Optional Hook**: {extension}
Command: `/{command}`
Description: {description}
```

```
Prompt: {prompt}
To execute: `/{command}`
```
```

- **Mandatory hook** (`optional: false`):

```
Extension Hooks
```

```
Automatic Hook: {extension}
Executing: `/{command}`
EXECUTE_COMMAND: {command}
```

After emitting the block above you MUST actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal `{command}` id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently